Part 01

1. Why does defining a custom constructor suppress the default constructor in C#?

When you define any constructor yourself, C# does not automatically provide the parameterless default constructor. If you need a parameterless constructor, you must define it explicitly.

2. How does method overloading improve code readability and reusability?

Method overloading allows multiple methods to have the same name but different parameters. This makes code easier to understand and reuse because the same operation can be performed with different types or numbers of inputs.

3. What is the purpose of constructor chaining in inheritance?

Constructor chaining allows a derived-class constructor to call a base-class constructor. It ensures that the inherited properties are properly initialized and avoids repeating initialization code.

4. How does new differ from override in method overriding?

new **hides** the base-class method, while `override` **redefines** a virtual or abstract base method using polymorphism. With `override`, the derived version is called even when the object is accessed through a base-class reference.

5. Why is ToString() often overridden in custom classes?

`ToString()` is overridden to provide a meaningful string representation of an object instead of the default class name. This makes debugging, logging, and displaying object information easier.

6. Why can't you create an instance of an interface directly?

An interface only defines a contract; it does not provide a complete implementation of an object. Therefore, it cannot be instantiated directly. A class that implements the interface must be instantiated instead.

7. What are the benefits of default implementations in interfaces introduced in C# 8.0?

Default interface implementations allow developers to add new methods to an existing interface without forcing every implementing class to immediately implement them. This improves **backward compatibility** and makes interfaces easier to evolve.

8. Why is it useful to use an interface reference to access implementing class methods?

An interface reference allows us to work with different classes through a common contract. This supports **polymorphism, loose coupling, flexibility, and easier testing and maintenance**.

9. How does C# overcome the limitation of single inheritance with interfaces?

C# allows a class to inherit from only one base class, but it can implement multiple interfaces. Therefore, a class can get behavior contracts from multiple interfaces without requiring multiple class inheritance.

10. What is the difference between a virtual method and an abstract method in C#?

A **virtual method** has a default implementation in the base class and can optionally be overridden by a derived class.

An **abstract method** has no implementation in the base class and must be implemented by a non-abstract derived class.

Part 02

1. What is the difference between a class and a struct in C#?

The main difference is that a **class is a reference type**, while a **struct is a value type**.

- **Class:** Objects are reference types and are typically allocated on the heap.
- **Struct:** Variables contain the value directly and are value types.
- A class supports inheritance from another class, while a struct cannot inherit from another struct or class.
- Structs are generally suitable for small, lightweight data types, while classes are better for objects with more complex behavior and identity.

2. If inheritance is a relation between classes, clarify other relations between classes.

The main relationships between classes are:

- **Inheritance (IS-A):** One class derives from another class.
  Example: `Car IS-A Vehicle`.
- **Association (USES/KNOWS-A):** One class is related to or uses another class.
  Example: `Teacher` works with `Student`.
- **Aggregation (HAS-A):** One class contains another class, but the contained object can exist independently.
  Example: `Department HAS-A Teacher`.
- **Composition (OWNS-A):** One class strongly owns another object, and the contained object usually depends on the owner for its lifecycle.
  Example: `House HAS-A Room`.
- **Dependency (DEPENDS-ON):** One class temporarily depends on another class to perform an operation, such as receiving it as a method parameter.